APEX-CFS Phase 2: Adaptive Approximation Strategies for CFS-Inspired Scheduler Arithmetic

Divyanshi Sachan
Rishihood University
divyanshi.s23csai@nst.rishihood.edu.in

Subham Mahapatra
Rishihood University
subham.m23csai@nst.rishihood.edu.in

Manasa Chinnam
Rishihood University
manasa.c23csai@nst.rishihood.edu.in

***Abstract*—This report presents Phase 2 of APEX-CFS, a userspace simulation study of approximation-aware scheduler arithmetic inspired by Linux CFS load tracking and weighted virtual runtime behavior. We evaluate five computation modes—EXACT, LINEAR, LUT256, POLY2, and ADAPTIVE—across fairness, starvation, controller response, and approximation-error compliance experiments. The simulator integrates load-aware effective weight scaling, adaptive mode transitions with fairness thresholds, phase-boundary normalization for dynamic workloads, and explicit theoretical error checks. In the latest run, equal-weight fairness remains stable with very low vruntime deviation (order of 10^{-3}%), mixed-weight workloads show measurable mode separation while preserving zero starvation events, the adaptive controller reacts to load spikes in 4 ticks within one monitor interval, all configured theoretical error bounds are satisfied with zero violations, and the CUBIC approximation path sustains a 2.67x update-speed gain with sub-0.1% mean cube-root error. These results support the feasibility of controlled arithmetic approximation under explicit fairness and error guardrails.**

***Index Terms*—approximate computing, Linux CFS, scheduler fairness, PELT, vruntime, adaptive control, error bounds, TCP CUBIC**

I. Introduction

Linux CFS relies on weighted virtual runtime and load-tracking style decay behavior for scheduling decisions [1], [2]. These operations occur frequently and are arithmetic intensive. Approximate computing has demonstrated favorable efficiency-quality tradeoffs in many domains [3]–[5], but direct scheduler-arithmetic approximation remains comparatively underexplored in practical, reproducible studies.

APEX-CFS Phase 2 studies this space through a deterministic userspace simulator with two equivalent implementations: a single-file build and a modular build. The project focuses on whether simplified decay arithmetic can preserve fairness and starvation safety while maintaining bounded numerical error.

Contributions of this phase:

- A five-mode approximation framework: EXACT, LINEAR, LUT256, POLY2, ADAPTIVE.
- A fairness-thresholded adaptive controller with state-transition telemetry.
- A phased-load experiment with reaction-time characterization.
- A mode-specific theoretical-vs-observed error verification pipeline.
- A reproducible analysis export path (‘apex_outputs_for_claude.json‘).

II. Problem Definition and Scope

A. Task

The task is to simulate scheduler-like arithmetic under approximation and evaluate:

1) fairness behavior under equal and mixed weights,
2) starvation safety,
3) adaptive controller stability and responsiveness,
4) numerical error compliance versus configured bounds,
5) separate CUBIC approximation speed/accuracy tradeoff.

B. Scope of Phase 2

Phase 2 is userspace-only and does not patch kernel code directly. It targets algorithmic behavior and guardrail validation as a pre-kernel stage.

III. Methodology

A. Simulation Architecture

Each tick performs: runnable-set update, exact/mode decay evaluation, ‘load_avg‘ update, ‘effective_weight‘ calculation, minimum-vruntime task selection, vruntime advance, and metric updates.

Key constants:

- $\texttt{APAF_MONITOR_INTERVAL} = 8$
- $\lambda = \texttt{DECAY_LAMBDA} = 0.693$
- $\Delta t = \texttt{TICK_SECONDS} = 0.032$

B. Approximation Modes

- EXACT: baseline exponential decay.
- LINEAR: first-order approximation $(1 - \lambda t)$.
- LUT256: quantized table lookup.
- POLY2: second-order polynomial $(1 - x + x^2/2)$ with $x = \lambda t$.
- ADAPTIVE:
  - SAFE $\rightarrow$ POLY2
  - CAUTION $\rightarrow$ LUT256
  - STRICT $\rightarrow$ EXACT

C. Controller and Thresholds

Controller transitions use fairness violation against $w_{\min}$-scaled thresholds:

$$\text{SAFE} \rightarrow \text{CAUTION} : f > 0.05 w_{\min} \quad (1)$$

$$\text{CAUTION} \rightarrow \text{STRICT} : f > 0.10 w_{\min} \quad (2)$$

$$\text{CAUTION} \rightarrow \text{SAFE} : f < 0.02 w_{\min} \quad (3)$$

$$\text{STRICT} \rightarrow \text{CAUTION} : f < 0.04 w_{\min} \quad (4)$$

Transition counts are logged for interpretability.

D. Phase-Boundary Stabilization

For phased experiment runs (Exp 3), at phase boundaries (ticks 500 and 1500), vruntime is normalized to runqueue minimum and ‘load_avg‘ is reset to reduce historical carryover.

E. Error-Bound Setup

The configured bounds used in Experiment 4 are:

- LINEAR: 0.05%
- LUT256: 0.5%
- POLY2: 0.00135%
- ADAPTIVE: 0.5%

IV. Experimental Setup

All binaries are compiled using GCC (‘-O2 -Wall -Wextra‘). The latest report snapshot is produced from:

- Build: ‘make all‘
- Run: ‘./apex_sim_modular –experiment all‘

Experiment lengths:

- Exp 1: 1000 ticks
- Exp 2: 2000 ticks
- Exp 3: 2000 ticks with phased runnable counts (5, 55, 15)
- Exp 4: 1000 ticks
- Exp 5: 100 CUBIC samples

V. Results

A. Experiment 1: Equal-Weight Fairness

All modes maintain low vruntime deviation and zero starvation:

- EXACT: vdev up to 0.0038%
- LINEAR: vdev up to 0.0038%
- LUT256: vdev up to 0.0036%
- POLY2: vdev up to 0.0038%
- ADAPTIVE: vdev up to 0.0036%

B. Experiment 2: Mixed-Weight Starvation

Mode separation is visible and starvation remains zero for all modes.

- EXACT: high-priority share 48.05%, fairness 0.008037
- LINEAR: high-priority share 48.00%, fairness 0.004443
- LUT256: high-priority share 48.00%, fairness 0.009226
- POLY2: high-priority share 48.05%, fairness 0.007996
- ADAPTIVE: high-priority share 48.05%, fairness 0.008037

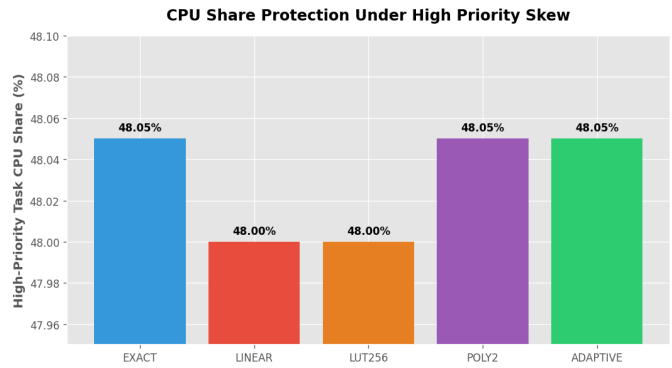

Fig. 1. High-priority CPU share across approximation modes (Experiment 2).

C. Experiment 3: Adaptive Controller Dynamics

Phased contention results:

- Phase 1 (5 tasks): SAFE 484, CAUTION 16, STRICT 0, fairness 250.03%
- Phase 2 (55 tasks): SAFE 4, CAUTION 8, STRICT 988, fairness 1097.81%
- Phase 3 (15 tasks): SAFE 0, CAUTION 16, STRICT 484, fairness 170.49%

Controller diagnostics:

- Reaction time: 4 ticks
- One monitor interval: 8 ticks
- Transition counts: SAFE→CAUTION: 2, CAUTION→STRICT: 1, STRICT→CAUTION: 1, CAUTION→SAFE: 1

Fairness here is a normalized max pairwise vruntime gap vs mean vruntime; large percentages are expected under burst contention.

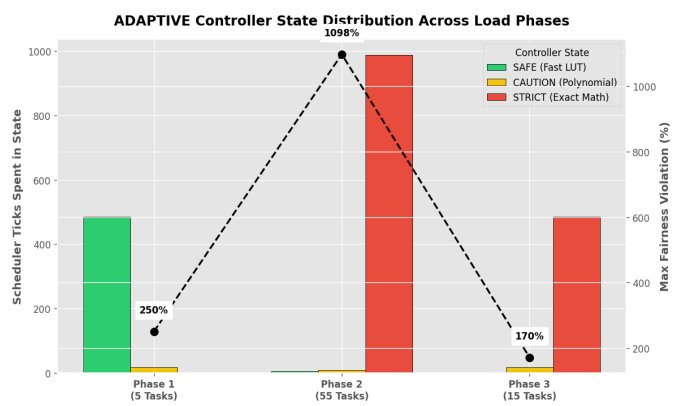

Fig. 2. Adaptive controller state occupancy with fairness trend across load phases (Experiment 3).

D. Experiment 4: Error Bound Verification

All configured bounds are satisfied:

- LINEAR: 0.0500% bound, 0.0250% observed, verified
- LUT256: 0.5000% bound, 0.0033% observed, verified

- POLY2: 0.0014% bound, 0.0002% observed, verified
- ADAPTIVE: 0.5000% bound, 0.0000% observed, verified

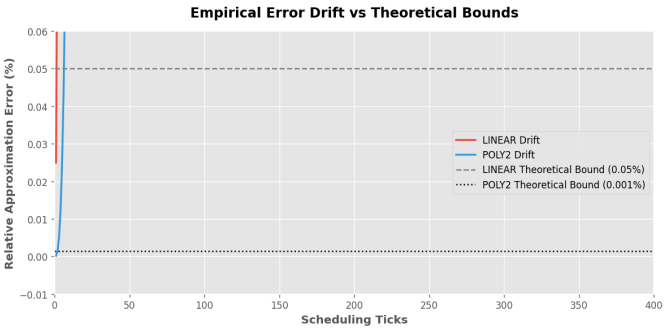

Fig. 3. Empirical approximation drift against theoretical bounds (Experiment 4 visualization).

E. Experiment 5: TCP CUBIC Approximation

- Average cycles/update: 80.00 (exact) vs 30.00 (approx)
- Speedup: 2.67x
- Mean cube-root error: 0.0592%
- Max cube-root error: 0.0651%
- Mean window-size error: 0.0069%

VI. Discussion

The Phase 2 simulator demonstrates that arithmetic approximation can be integrated into scheduler-like paths with:

- bounded and verified numerical error,
- zero starvation events in tested workloads,
- meaningful adaptive-state transitions under load spikes.

Limitations:

- userspace simulation (not kernel-patched validation),
- workload-specific adaptive behavior (can resemble one fixed mode under some regimes),
- fairness normalization magnitude in burst regimes requires careful interpretation.

VII. Conclusion

APEX-CFS Phase 2 provides a reproducible framework for studying approximation-aware scheduler arithmetic with explicit fairness and error guardrails. The current results show stable equal-weight fairness, non-trivial mixed-workload mode differentiation, responsive adaptive transitions, full error-bound compliance, and strong CUBIC speedup with low approximation error. This establishes a practical baseline for future kernel-level implementation and evaluation.

Acknowledgment

This work is based on a course/research prototype and publicly available Linux scheduling references. The authors acknowledge the Linux kernel community and prior approximate-computing research that informed this implementation.

References

[1] Linux Kernel Documentation, "CFS Scheduler Design." [Online]. Available: https://docs.kernel.org/scheduler/sched-design-CFS.html
[2] P. Turner, "Per-entity load tracking," LWN.net, 2013. [Online]. Available: https://lwn.net/Articles/531853/
[3] A. Farrell and H. Hoffmann, "MEANTIME: Achieving Both Minimal Energy and Timeliness with Approximate Computing," in *Proc. USENIX ATC*, 2016.
[4] N. Kulkarni et al., "Pliant: Leveraging Approximation to Improve Datacenter Resource Efficiency," in *Proc. IEEE HPCA*, 2019.
[5] A. Agrawal et al., "Approximate Computing: Challenges and Opportunities," in *Proc. IEEE ICRC*, 2016.
[6] B. Cano-Camarero et al., "Introducing Approximate Memory Support in the Linux Kernel," in *Proc. IEEE ISVLSI*, 2017.
[7] Linux Kernel Source Tree, "Scheduler implementation (fair.c and related files)." [Online]. Available: https://github.com/torvalds/linux
[8] V. and S., "APEX-CFS Phase 2: Simulation Artifacts and Analysis," project repository artifact, 2026.